

- 7 Uvod u napredno programiranje
 - 7.1 Rekurzivne procedure i funkcije
 - 7.1.1 Dinamičke strukture podataka
 - 7.1.1.1 Vezane liste
 - 7.1.1.2 Binarno stablo
 - 7.2 MODEL RESURSA ZA APLIKACIJE TEMELJENE NA PRIMJENI INTERNETA (WEB APLIKACIJE)
 - 7.2.1 UVOD
 - 7.2.2 ZAHTJEVI ZA TROSLOJNE ARHITEKTURE
 - 7.2.3 MODEL TRANSAKCIJA
 - 7.2.3.1 Definicija transakcije
 - 7.2.3.2 Svojstva transakcija
 - 7.2.4 MICROSOFT PODRŠKA DISTRIBUIRANIM TRANSAKCIJAMA
 - 7.2.5 KOMPONENTE U ČVRSTO VEZANIM SUSTAVIMA (TIGHTLY COUPLED)
 - 7.2.6 SIGURNOST I PRAĆENJE DOGAĐAJA
 - 7.2.7 KOMPONENTE U SLABO VEZANIM SUSTAVIMA (LOOSELY COUPLED)
 - 7.2.8 ZAKLJUČAK: EVOLUCIJA PREMA WEB USLUGAMA
 - 7.3 Perl - jezik za dinamičko generiranje web stranica
 - 7.3.1 Svojstva jezika Perl
 - 7.3.2 Usporedba jezika Perl i C
 - 7.3.2.1 Razina jezika
 - 7.3.2.2 Nizak stupanj tipizacije
 - 7.3.3 Generiranje izvještaja u obliku html-a pomoću jezika Perl, na primjeru studentske oglasne ploče
 - 7.3.3.1 Podaci
 - 7.3.3.2 Predlošci za generiranje oglasa
 - 7.3.3.3 Ulazni parametri korisnika (rubrika i podrubrika; html):
 - 7.3.3.4 Generiranje stranice oglasa (kod u Perl-u)
 - 7.3.3.5 Izlazni html dokument
 - 7.4 C#

7 UVOD U NAPREDNO PROGRAMIRANJE

7.1 Rekurzivne procedure i funkcije

U potprogramu (FUNCTION ili PROCEDURE) mogu se pozivati drugi potprogrami koji su prethodno deklarirani. Ako potprogram poziva samog sebe, onda za njega kažemo da je rekurzivan. Rekurzivni potprogram zahtijeva više memorijskog prostora i više vremena od odgovarajućeg nerekurzivnog potprograma.

Rekurzivne funkcije i procedure omogućuju jednostavno i efektno rješavanje problema koji su prirodno rekurzivni i nespretno ih je rješavati bez rekurzije.

Kod rješavanja rekurzivnog problema, problem se svodi na novi problem sličan polaznom, ali nešto jednostavniji. Ovaj postupak se nastavlja sve dok rješenje problema ne postane trivijalno.

Primjeri:

1. Potenciranje cijelog broja m prirodnim eksponentom n možemo definirati ovako:

- ako je $n = 1$ tada je $p = m^1 = m$

- inače je $p = m^n = m^{n-1} * m$

FUNCTION mnan (m : integer ; n : prirodni) : integer;

BEGIN

IF $n = 1$ THEN mnan := m

ELSE mnan:= mnan($m, n-1$) * m

END

Analizirajmo poziv funkcije mnan (5,3):

mnan (5,3) inicijalizira parametre $m=5$, $n=3$, vrijednost će biti mnan (5,2) * m ;

mnan (5,2) inicijalizira parametre $m=5$, $n=2$, vrijednost će biti mnan (5,1) * m ;

mnan (5,1) inicijalizira parametre $m = 5$, $n = 1$,

te je mnan (5,1)=5; Proces se vraća natrag

mnan (5,2) = mnan (5,1) * $m = 5 * 5 = 25$

mnan (5,3) = mnan (5,2) * $m = 25 * 5 = 125$

Odgovarajuća funkcija u C++ :

```
int mnan (int m, int n)
```

```
{
```

```
if (n == 1) return m;
```

```
else return mnan(m,n-1) * m;
```

```
}
```

2. Prethodnu funkciju možemo napraviti i bez rekurzije:

```
FUNCTION mnan (m : integer; n : prirodni) : integer;
VAR i,potencija : integer;
BEGIN
    potencija := m;
WHILE n > 1 DO
BEGIN
    potencija := potencija * m;
    n := n - 1
END;
mnan := potencija
END;
```

Odnosno, u C++ :

```
int mnan (int m, int n)
{
int potencija;
potencija = m;
    while (n > 1){
        potencija = potencija * m;
        n = n-1;
    }
    return potencija;
};
```

Bit rekurzivnih definicija je ista: najprije se navode vrijednosti funkcije za neke početne vrijednosti varijabli, a zatim se vrijednost funkcije za danu opću vrijednost varijabli izražava u zavisnosti od prethodno izračunatih vrijednosti funkcije koje odgovaraju manjim vrijednostima varijabli.

Za funkciju faktorijel ($f(a) = n!$) imamo:

```
FUNCTION f(n: CARDINAL) : CARDINAL ;
IF n < 2 THEN f := 1
ELSE f := n * f(n-1)
END_IF
END FUNCTION
```

Odnosno, u C++ :

```
int fakto (int n){
    if (n < 2) return 1;
    else
        return n*fakto(n-1);
}
```

Za razumijevanje rekurzivnih potprograma središnje pitanje je što se u stvari događa kad potprogram poziva samog sebe. U tom procesu glavnu ulogu igra mehanizam "steka" tj. memorijskog elementa iz koga se najprije učitava onaj sadržaj koji je u njega posljednji upisan (LIFO). Uobičajeni princip realizacije rekurzivnog potprograma je slijedeći:

1. Rekurzivni potprogram poziva se sa određenom početnom vrijednošću varijable (npr. neka je $f(6)$ poziv u slučaju faktoriijela).
2. Računski proces unutar potprograma odvija se nesmetano sve do točke kada dolazi do poziva istog potprograma sa izmijenjenom (najčešće smanjenom) vrijednošću argumenta ($f(5)$ u slučaju faktoriijela).
3. Novi poziv istog potprograma dovodi do privremenog prekida izvršavanja tekućeg poziva rekurzivnog potprograma. Pri tome se najprije sve vrijednosti lokalnih varijabli spremaju u "stek".
4. Ponavljanjem procesa opisanog u točkama 1, 2 i 3 stalno raste sadržaj "steka" sve dok se ne stigne do krajnjeg uvjeta koji više ne zahtijeva rekurzivne pozive potprograma (u slučaju faktoriijela to je $f(1)$).
5. Povratkom sa nižeg nivoa izvršavanja rekurzivnog potprograma na viši nivo nastavlja se računski proces koji je bio prekinut u točki 3. Učitavaju se i obnavljaju vrijednosti lokalnih varijabli koje su bile pohranjene u "stek".
6. Kada je dobiven pretposljedni rezultat rekurzivnog potprograma "stek" je doveden u početno stanje, izračunava se krajnja vrijednost rezultata i dostavlja pozivajućem programu. Time je rad rekurzivnog potprograma uspješno završen.

Rekurzivne procedure mogu se realizirati na način koji je analogan sa načinom koji je primjenjen kod rekurzivnih funkcija. Kao najjednostavniji primjer navodi se slijedeća procedura koja prikazuje prirodne brojeve od 1 do n :

```
PROCEDURE DISPLAY (n : CARDINAL);  
IF n > 0 THEN  
  DISPLAY (n-1);  
END_IF  
END PROCEDURE
```

C++ ne poznaje procedure, ali odgovarajući učinak će imati funkcija označena s *void* :

```
void display (int n)
{
  if (n > 0){
    display (n-1);
  }
}
```

7.1.1 Dinamičke strukture podataka

7.1.1.1 Vezane liste

Pokazivači se najčešće upotrebljavaju za stvaranje dinamičke vezane liste (popisa) podataka. Svaki član liste je zapis (RECORD) kojem je dodano jedno (može i više) polje koje je pokazivač nekog člana liste, odnosno koje sadrži adresu slijedećeg člana liste. Za ulaz u listu potreban je jedan glavni pokazivač koji pokazuje prvi član u listi i mora biti poznat izvan liste. Promjene u listi vršimo promjenom pokazivača.

```
TYPE pokazivac = ^element;
element = RECORD
    veza : pokazivac;
    broj : integer
END;
```

```
VAR novi, glava, n, p, q : pokazivac;
```

U C++ element liste definiramo kako slijedi:

```
struct element
{
    element *veza;
    int broj;
};
```

dok odgovarajuće pokazivače definiramo korištenjem znaka `""` :

```
element *novi, *glava, *n, *p, *q;
```

Kreirajmo listu brojeva tako da novi broj dodajemo na početak liste:

```
glava := NIL; (* prvi pokazivac *)
REPEAT
  write (' broj - - > '); readln (x);
  new (novi);
  novi^.broj := x;
  novi^.veza := glava;
  glava := novi
UNTIL x = 9;
Odnosno, u C++:
```

```
glava = NULL; // prvi pokazivac
```

```

do
{
cout <<" broj - - > "; cin >> x;
novi = new (element);
novi -> broj = x;
novi -> veza = glava;
glava = novi;
}while (x != 9);

```

Listu možemo ispisati tako da pokazivačem p prolazimo kroz listu i ispisujemo komponentu broj:

```
p := glava; (* pokazivač na početak liste *)
```

```

WHILE p < > NIL DO
BEGIN
write (p^.broj);
p := p^.veza (* nova komponenta liste *)
END;

```

što bi u C++ izgledalo ovako :

```

p = glava; // pokazivač na početak liste

while (p != NULL){
    cout << p -> broj << endl;
    p = p -> veza; //nova komponenta liste
};

```

7.1.1.2 Binarno stablo

Binarno stablo je hijerarhijska struktura podataka u kojoj svaki nadređeni element može imati najviše dva podređena elementa. Elemente stabla nazovimo čvorovima. Binarno stablo možemo definirati aksiomatski:

1. prazan skup čvorova je binarno stablo,
2. ako je C čvor, $S1$ i $S2$ binarna stabla, tada je uređena trojka $(S1, C, S2)$ binarno stablo.

Neka je:

```

TYPE veza = ^stablo;
stablo = RECORD
    broj : integer;
    desno : veza;
    lijevo : veza;
END;

```

```
VAR korijen, p : veza;
```

C++:

```
struct stablo
```

```
{
    int broj;
    stablo *desno;
    stablo *lijevo;
};
```

```
stablo *korijen, *p;
```

Ispis vrijednosti čvorova :

```
PROCEDURE izlazl(p : veza);
BEGIN
    IF p < > NIL THEN
        BEGIN
            izlazl (p^.lijevo);
            write(p^.broj:3);
            izlazl (p^.desno)
        END
    END; (* izlazl *)
```

C++:

```
void izlazl (stablo *p);
{
    if (p != NULL){
        izlazl (p -> lijevo);
        cout << p -> broj << endl;
        izlazl (p -> desno);
    }
}; //izlazl
```

```
PROCEDURE novicvor(VAR scvor : veza; n : integer);
VAR cvor : veza;
BEGIN (* novicvor *)
    new(cvor);
    WITH cvor^ DO
        BEGIN
            broj : = n;
            desno : = NIL;
            lijevo : = NIL
            IF broj > scvor^.broj THEN
                scvor^.desno = cvor
            ELSE
                scvor^.lijevo = cvor;
        END
    END; (* novicvor *)
```

C++ - funkcija kreira novi čvor i vraća njegovu adresu:

```
void novicvor(stablo *scvor; int n);
{ // novicvor
    stablo *cvor;
    cvor = new(stablo);
    cvor -> broj = n;
```

```

cvor -> desno = NULL;
cvor -> lijevo = NULL;
if (cvor -> broj > scvor -> broj)
{
    scvor -> desno = cvor;
};
else
{
    scvor -> lijevo = cvor;
};
}; // novicvor

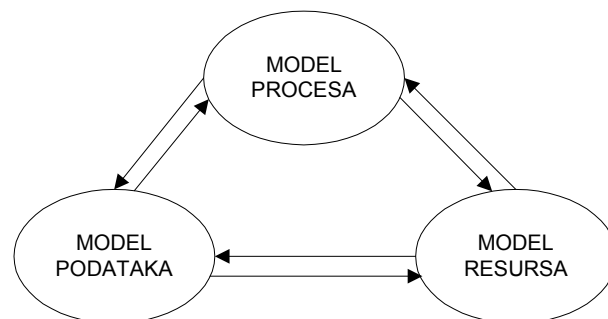
```

7.2 MODEL RESURSA ZA APLIKACIJE TEMELJENE NA PRIMJENI INTERNETA (WEB APLIKACIJE)

7.2.1 UVOD

Poslovni sustavi orijentirani na primjenu Interneta razvijaju informacijske sustave temeljene na webu. Web aplikacije karakteriziraju slabo povezani slojevi replicirajućih, strogo profiliranih programskih sustava.

Model resursa je, općenito, usko povezan s modelom podataka i modelom procesa i svi zajedno čine jedinstveni model informacijskog sustava (Slika 1.).



Slika 1. Model informacijskog sustava

Sa stajališta programiranja, ključni dio modela resursa su softverske komponente, sustavi i infrastrukture.

Softverske komponente omogućuju jeftinu i brzu izgradnju infrastrukture za skalabilne, dostupne, sigurne i upravljive višeslojne Web aplikacije.

Općenito se za Internet aplikacije (telematičke aplikacije) razvijaju tri osnovna sloja:

- korisnički sloj (*user services*),
- poslovni sloj (*business services*)
- podatkovni sloj (*data services*).

Takve aplikacije nazivamo **troslojne aplikacije**.

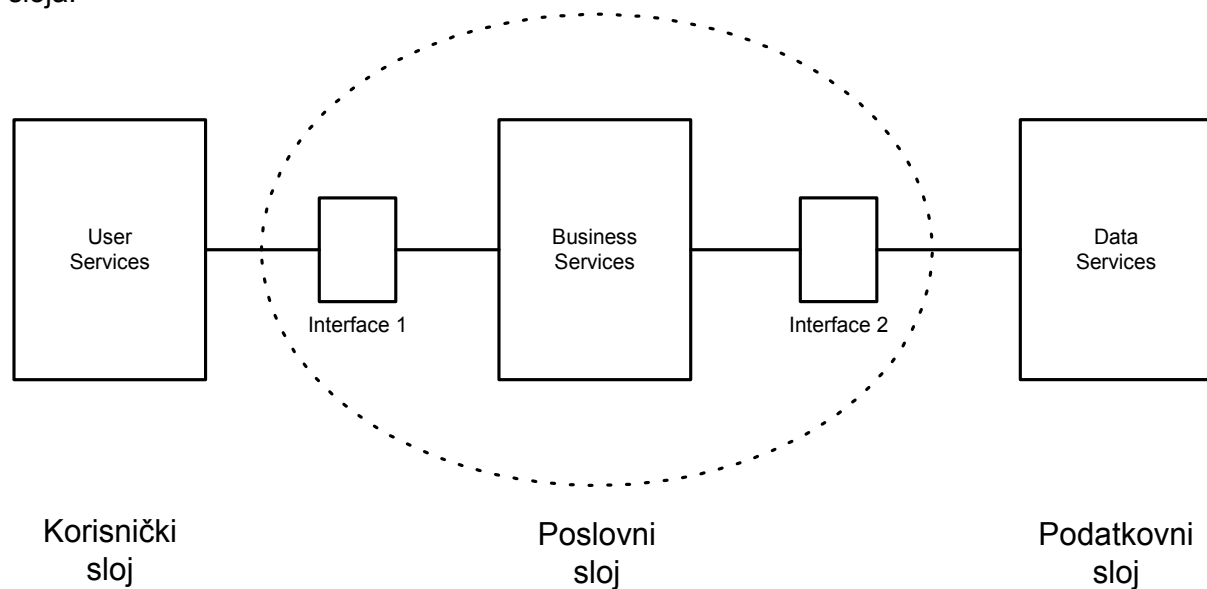
7.2.2 ZAHTJEVI ZA TROSLOJNE ARHITEKTURE

Arhitektura potrebna za izradu složenih poslovnih sustava baziranih na Web aplikacijama mora ispuniti sljedeće uvjete:

- Linearna **skalabilnost** – mogućnost kontinuiranog rasta sustava kako bi zadovoljio korisničke zahtjeve i zahtjeve poslovnih procesa
- Kontinuirana **dostupnost** usluga – upotreba redundantnih komponenti i specijalizacije funkcija koje su otporne na greške
- **Sigurnost** podataka i infrastrukture – zaštitom podataka i infrastrukture od neprijateljskih napada ili krađe
- Jednostavnost i potpunost **upravljivosti** – čime sustavi osiguravaju da operativnost bude efikasna unatoč rastu sustava.

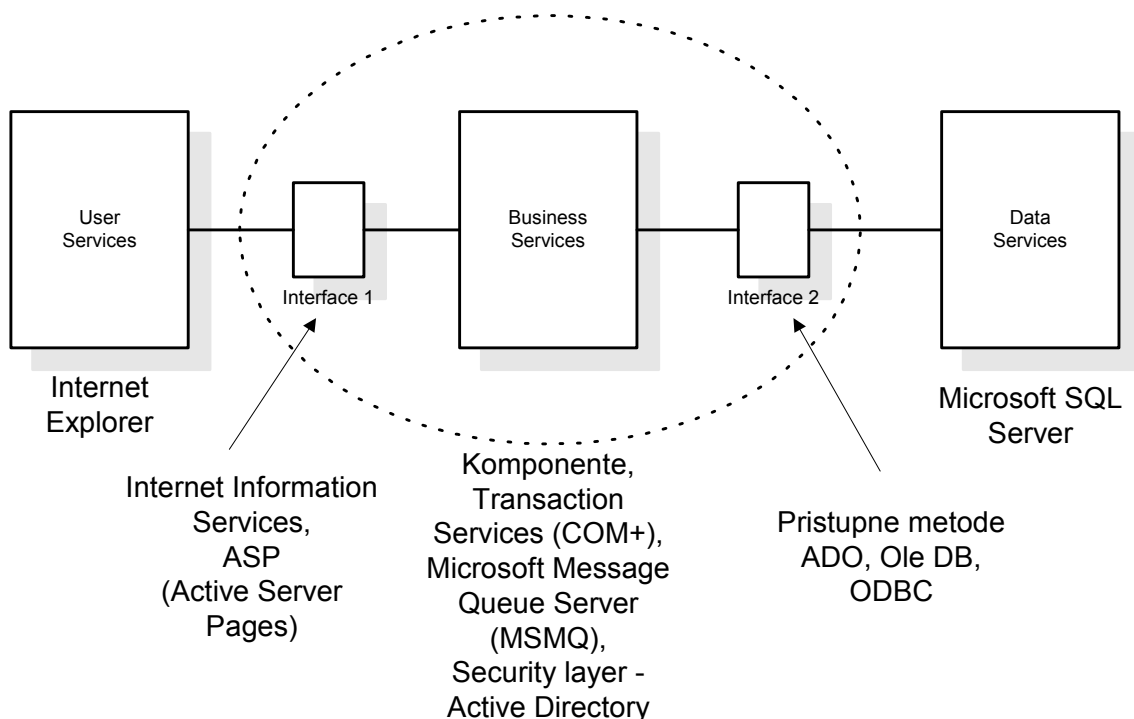
Podrška za troslojne aplikacije zahtijeva postojanje već opisana tri sloja ali i dva sučelja (slika 2.):

- sučelje 1. između korisničkog i poslovnog sloja,
- sučelje 2. između poslojnog i podatkovnog sloja.

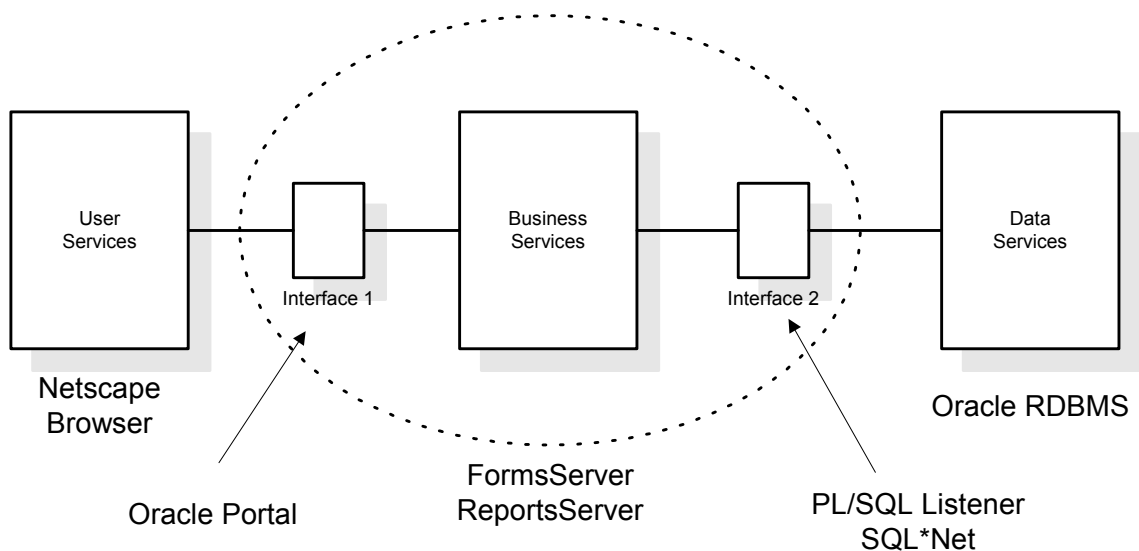


Slika 2. Opći model troslojne aplikacije

Vodeći svjetski proizvođači programske podrške su Microsoft i Oracle te za njih možemo prikazati i analizirati softverske platforme za razvoj troslojnih aplikacija (slike 3. i 4.).



Slika 3. Programska platforma za Web aplikaciju u Microsoft tehnologiji



Slika 4. Programska platforma za Web aplikaciju u Oracle tehnologiji

7.2.3 MODELI TRANSAKCIJA

7.2.3.1 Definicija transakcije

U literaturi se navode različite definicije transakcija, među kojima su sljedeće:

“Transakcija je niz poslova (operacija) koja u cijelosti uspije ili cijelosti ne uspije, bez obzira na broj koraka od kojih se sastoji (na primjer naručivanje, provjera stanja na skladištu ili plaćanje)”.

“Transakcija je jedinica posla koja se obavlja kao atomička operacija – to jest, operacija uspije kao cjelina ili ne uspije kao cjelina”.

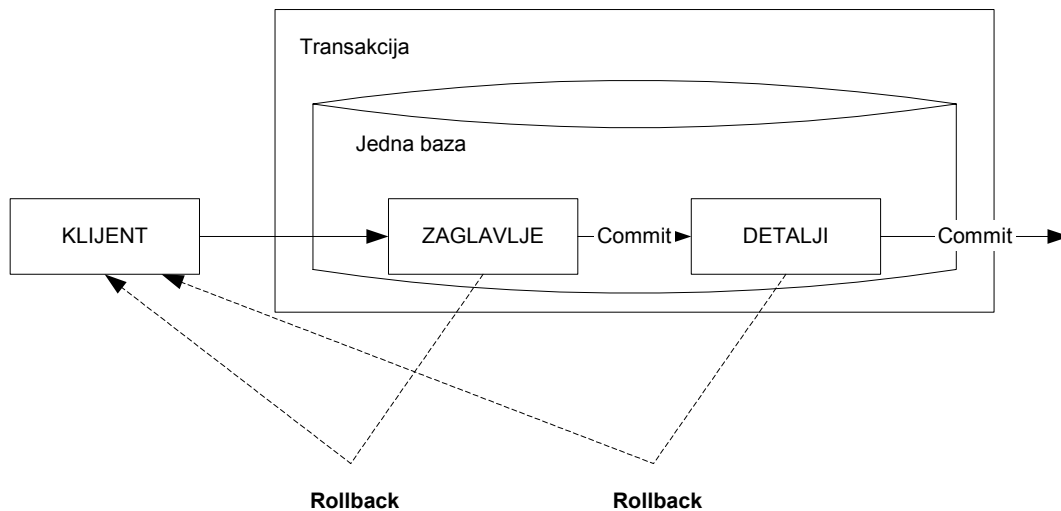
Kada je riječ o informatičkim poslovnim sustavima, nužno je razlikovati dvije vrste transakcija:

- poslovne transakcije
- transakcije informacijskog sustava.

Važno je uočiti da se te dvije vrste transakcije nužno ne poklapaju, stoga je u razvoju poslovnih sustava potrebno strogo definirati opseg svake vrste i njihova preklapanja.

7.2.3.2 Svojstva transakcija

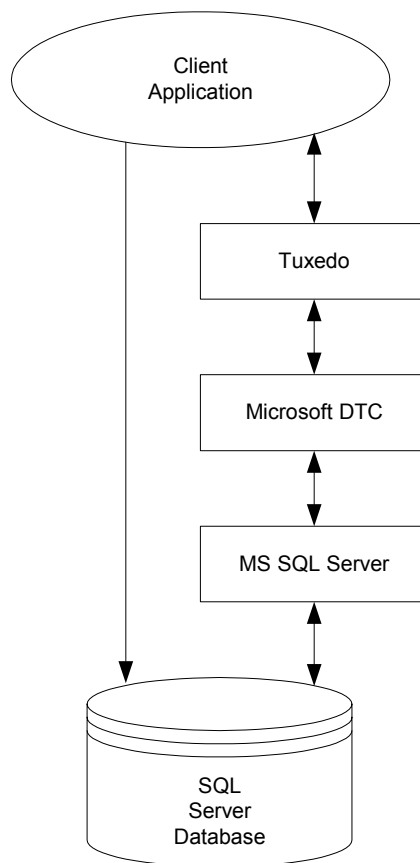
Najjednostavniji primjer transakcije nalazimo kod rada s bazama podataka. Npr. u slučaju zapisa podataka u dvije tablice. Slika 5 predstavlja jedan od mogućih scenarija: zapisivanje zaglavlja i detalja (stavaka) dokumenta u bazu podataka. Prvo se zapisuju podaci u tablicu ZAGLAVLJE, pa ukoliko je zapisivanje uspješno, onda u tablicu DETALJI. Budući se taj čitav proces nalazi “omotan” u transakciju, ukoliko se dogodi da iz nekog razloga ne uspije zapisivanje u tablicu ZAGLAVLJE ili u tablicu DETALJI, operacija se poništava i rekonstruira se prethodno stanje sustava (*Rollback*). Ukoliko je bez greške prošlo zapisivanje podataka i u jednu i u drugu tablicu, transakcija je uspješna i podaci su došli u stabilno stanje, odnosno potvrđeno je njihovo zapisivanje.



Slika 5. Model transakcije

7.2.4 MICROSOFT PODRŠKA DISTRIBUIRANIM TRANSAKCIJAMA

Distribuirane transakcije povezuju dva ili više poslužitelja, odnosno upravljača resursima (*resources manager*). Upravljanje transakcijama se koordinira između upravljača resursa pomoću poslužitelja koji se zove nadglednik odvijanja transakcija (Transaction Processing Monitor – TPM). TPM je sistemski servis namjenjen integraciji managera resursa u višeslojnoj arhitekturi i na Windows DNA platformi podržava X/Open XA specifikaciju koja omogućuje višeplatformsku operabilnost (slika 6).



Slika 6. Model procesiranja transakcije

Na Windows DNA platformi postoji takav servis koji se zove Microsoft Distributed Transaction Coordinator (MS DTC).

Na nivou aplikacije, upravljanje distribuiranom transakcijom se odvija na sličan način kao da se radi o lokalnoj transakciji na jednom poslužitelju. Na kraju transakcije, aplikacija zahtjeva da se potvrdi, zapiše operacija ili da se rekonstrira prethodno stanje. Upravo tu, kod potvrđivanja, odnosno zapisivanja (*commit*) postoji osnovna razlika u upravljanju transakcijom, kako bi se smanjio rizik da zbog greške na mreži dođe do slučaja da jedan manager resursa uspije zapisati podatke, dok drugi restaurira prethodno stanje (*rollback*). Da se to ne bi dogodilo, proces uspješnog odvijanja transakcije (zapisivanja podataka) odvija se u dvije faze (pripremnoj fazi i fazi zapisivanja), a sam proces se naziva “2-phase commit” (2PC).

7.2.5 KOMPONENTE U ČVRSTO VEZANIM SUSTAVIMA (TIGHTLY COUPLED)

Microsoftovi standardi za izradu komponenti i međuprocesnu komunikaciju COM I DCOM su rješenja za binarnu kompatibilnost i neovisnost izrade pojedinih komponenti sustava o programskom jeziku. Takve komponente imaju podršku za pakiranje, fragmentiranje i distribuciju funkcionalnosti sustava.

COM i DCOM se primjenjuju u sustavima koji podrazumjevaju stabilne i stalne veze, kao što su lokalne mreže ili globalne mreže sa širokim kanalima komunikacije, te protokole koji podržavaju pohranu stanja konekcije.

Mana tih standarda je u tome što protokol HTTP na intranet/Internet-u je tzv. *Connectionless* protokol, a sam princip pohrane stanja konekcije je previše "skup" za te medije komunikacije. Na Internetu se može očekivati više konkurentnih korisnika nego što je to slučaj u lokalnoj mreži. Pohrana stanja za svakog takvog korisnika zahtjevala bi povećane resurse, te bi time povećala troškove konfiguriranja i održavanja sustava.

7.2.6 SIGURNOST I PRAĆENJE DOGAĐAJA

COM+ podržava i sigurnost baziranu na pripadnosti sigurnosnoj grupi i kroz prava pristupa određenom procesu. U sigurnosti baziranoj na pripadnosti sigurnosnoj grupi, pristup određenom dijelu aplikacije je dozvoljen ili odbijen bazirano na logičkoj grupi ili ulozi kojoj pozivatelj pripada (npr. Administrator, programer ili računovođa). Novost je tzv. Personalizacija, odnosno prenošenje sigurnosnog konteksta od klijenta, preko komponenti srednjeg sloja pa sve do spremišta podataka (npr. SQL Server Roles).

Praćenje događaja u COM+ je moguće kroz sustav izdavača (*publishers*) i pretplatnika (*subscribers*). Izdavač šalje obavjest o događaju pretplatniku, a podaci o pretplatniku su poznati izdavaču preko tzv. Pretplate (*subscriptions*). Izdavač može biti bilo kakav objekt koji implementira metode na sučelju za dojavu događaja.

7.2.7 KOMPONENTE U SLABO VEZANIM SUSTAVIMA (LOOSELY COUPLED)

Zbog manje širine komunikacijskog kanala Internet sustavi i Web aplikacije ne podržavaju permanentne veze (*connectionless*). Usred potencijalno velikog broja istovremenih korisnika postavlja se zahtjev da aplikacije i komponente ne pohranjuju stanje (*stateless*), te da podržavaju automatsko otpuštanje komponenti. Stoga se pojavljuje problem interoperabilnosti – razmjene podataka i pozivi komponentama na različitim platformama.

Zbog svega toga, pristupilo se drugoj tehnici izrade komponenti: slabo vezanim komponentama. Ta tehnika podrazumjeva primjenu dizajna aplikacija u kojem su komponente pisane tako da minimalno ovise jedna o drugoj. Programi koji koriste slabo vezane komponente ne oslanjaju se na veliki broj globalnih varijabli, svojstava i rutina. Slabo vezivanje može spriječiti da neizvršavanje jedne komponente u aplikaciji nužno ne uzrokuje neizvršavanje ostalih komponenti.

Extensible Markup Language (XML), kao i HTML, omogućava označavanje dijelova dokumenta pomoću oznaka (tag-ova). Za razliku od HTML-a, pomoću XML tag-ova mogu se ne samo formirati format i način prikaza nekog dokumenta, nego i semantička i organizacijska struktura dokumenta.

Navedena svojstva omogućuju da se XML standard koristi prvenstveno za razmjenu podataka između:

- aplikacija na istoj platformi
- aplikacija na različitim platformama.

Međutim, i dalje ostaje slijedeći problem: svaka aplikacija, proizvođač, industrija ima svoj način zapisivanja podataka, pa je potrebno odabrati između:

- standardizacija zapisa podataka ili
- servisa za konverziju između raznih standarda.

Upotrebom XML protokola moguća je komunikacija između različitih platforma. BizTalk inicijativa omogućuje izmjenu poslovnih dokumenata između različitih poslovnih sustava i industrijskih grana. Ono što nedostaje je način za povezivanje aplikacija i komponenti na različitim platformama.

SOAP je jednostavni protokol za pristup objektima. On ne samo da omogućava povezivanje komponenti na različitim platformama, nego i korištenje postojeće Internet infrastrukture za direktnu komunikaciju bez ometanja od strane vatrozida (*firewalls*).

Osim toga, SOAP je (*connectionless*) protokol koji ne zahtjeva pohranu stanja konekcije, te je zato pogodniji.

Microsoft aktivno sudjeluje u izradi specifikacije SOAP protokola, te već postoji SOAP SDK (*Software Development Kit*) koji omogućava da COM komponenta poziva preko *proxy-ja* neku drugu komponentu (bilo COM, bilo CORBA ili dr.) koja podržava SOAP protokol.

7.2.8 ZAKLJUČAK: EVOLUCIJA PREMA WEB USLUGAMA

Negdje na početku razvoja Interneta, računalni stručnjaci su bili sretni kad su spojili više računala, omogućili razmjenu podataka među njima i komunikaciju elektroničkom poštom. S vremenom, poželjeli su statičnim tekstualnim podacima dodati dinamiku i kreiran je HTTP protokol koji je omogućio vizualizaciju putem World Wide Web-a i slikovnih i dinamičkih sadržaja.

Danas, samo prikazivanje i nuđenje sadržaja nije dovoljno: traži se interaktivnost, komunikacija s korisnikom. Problem je što koriste različite platforme, te su se morali postaviti novi standardi:

- XML standard za razmjenu podataka
- SOAP za povezivanje komponenti i funkcija preko intraneta/Interneta.

7.3 Perl - jezik za dinamičko generiranje web stranica

Programski jezik Perl (Perl je kratica od "**P**ractical **E**xtraction and **R**eport **L**anguage" - jezik za ekstrakciju teksta i pravljenje izvještaja) je danas jedan od najraširenijih jezika skripata. Prvu verziju izradio je Larry Wall 1986. Prvobitna namjena mu je bila obrada tekstualnih datoteka i kreiranje izvještaja, dok se danas uglavnom koristi za pisanje CGI skripata (CGI = Common Gateway Interface) na web serverima. Upravo zahvaljujući ovom zadnjem, Perl je danas najpopularniji jezik skripata (MCMillan, 1998., str 2.).

Perl je napisan u C-u ponajprije za potrebe UNIX sustava. Sintaksa Perl-a u značajnom je dijelu naslijeđena od C-a što čini prelazak s C-a na Perl jednostavnim. Osim toga, problemi kao što je manipulacija tekstualnim ulazom daleko se jednostavnije rješavaju u Perl-u, nego u C-u, pa je Perl primjenjiv tamo gdje se ne zahtijeva brzina C-a. Perl je pogodan za brz razvoj aplikacija, jer je implementiran kao interpreter, pa se ne troši vrijeme na prevođenje.

7.3.1 Svojstva jezika Perl

Perl nudi slijedeća dobra svojstva, koja ga čine danas najpopularnijim jezikom skripata :

- jednostavna manipulacija tekstualnih podataka,
- sličnost sintakse Perl-a s C-om,
- sadrži jednostavan objektni model koji omogućuje objektno orijentirano programiranje,
- može uključivati module pisane u drugim programskim jezicima, a također i generirati kod drugih programskih jezika, npr. HTML-a,
- prenosivost Perl-a je izuzetno dobra, tako da programi pisani u Perl-u rade na različitim operativnim sustavima,
- jednostavno se uključuje u druge programske jezike,
- veliki broj modula - biblioteka koje omogućuju izvršavanje najrazličitijih funkcija, među kojima je i dinamičko generiranje Web stranica, slanje upita bazama podataka u SQL-u i kodiranje podataka,
- fleksibilne strukture podataka.

S druge strane, nedostaci Perl-a su slijedeći:

- sporost u odnosu na C,
- izvršavanje programa u Perl-u kasni jer se program prevodi svaki puta kad se starta,
- programski kod u Perl-u je često teško čitljiv, odnosno, zamršen,
- fleksibilnost podatkovnih struktura u Perl-u rezultirala je povećanim utroškom memorije.

Prema tome, jezik Perl uključuje sva osnovna svojstva jezika skripata :

- vrlo je visoke razine, što ga čini prenosivim između različitih kompjutorskih strojnih i programskih platformi,
- slabo je tipiziran (u odnosu na C) - ista varijabla može se interpretirati kao podatak različitih tipova,
- primjenjiv je za povezivanje gotovih komponenti pisanih u drugim programskim jezicima,
- interpretira se, što doprinosi brzini razvoja aplikacija.

7.3.2 Usporedba jezika Perl i C

Programski jezik C predstavlja tipičan primjer visoko efikasnog sistemskog programskog jezika, dok je Perl visoko fleksibilan jezik skripata. S druge strane, Perl je dobrim dijelom nastao na osnovi C-a, od kojeg nasljeđuje sintaksu. Komparativnom analizom Perl-a i C-a došlo se do slijedeće tablice koja pokazuje odnos između ta dva programska jezika :

Programski konstrukti	C	Perl
sekvenca	{ }	{ }
selekcija	if, switch	if, unless, switch
iteracija	for, while, do-while	while, until, foreach
potprogrami	funkcije	funkcije
polja neograničene veličine	ne	da
komentar	/* i */	#
Ostala svojstva		
način implementacije	prevoditelj	interpreter
razina u broju strojnih instrukcija po naredbi	2,50	15

razina u broju naredbi po funkcijskoj točki	128	21
glavne prednosti	brzina	fleksibilnost, visoka razina

Tablica 2.5 : Usporedba C-a i Perl-a

U tablici 2.5 možemo vidjeti da se jezici C i Perl međusobno sasvim neznatno razlikuju po ugrađenim programskim konstruktima, ali je Perl fleksibilniji u rukovanju podacima (npr. polja neograničene dužine). Daleko je veća razlika u načinu implementacije (Perl se tipično interpretira, C prevodi) i u razini jezika, što upućuje na veću produktivnost programera u Perl-u u odnosu na onoga koji programira u C-u, dok je prednost C-a brzina.

7.3.2.1 Razina jezika

Razinu programskog jezika možemo izraziti dvojako :

- u broju strojnih instrukcija koje računalo mora izvršiti za jednu instrukciju zadanog višeg programskog jezika ili
- u broju instrukcija zadanog programskog jezika koje su potrebne da bi se realizirala jedna funkcijska točka u programu (Jones, 1996.), pri čemu metriku funkcijskih točaka možemo definirati kao sredstvo za mjerenje veličine i produktivnosti softvera. Osnovnu funkcijsku točku definiramo kao jednu poslovnu funkciju krajnjeg korisnika, kao što je npr. jedan upit za ulaz podataka. Broj funkcijskih točaka u programu utvrđujemo brojanjem ulaza, izlaza, upita i logičkih grupiranja podataka sadržanih u jednoj aplikaciji i određivanjem faktora složenosti svakog od njih (Van Doren, 1997.).

Prema tome, programski jezik je to više razine što više strojnih instrukcija je potrebno za izvršenje jedne instrukcije tog jezika, odnosno, što je manje instrukcija tog jezika potrebno za jednu funkcijsku točku.

Općenito se može reći da razina programskih jezika raste s njihovim razvojem, kao što možemo vidjeti u slijedećoj tabeli, preuzetoj od (Jones, 1996):

Jezik	Razina (broj strojnih naredbi po instrukciji)	Prosječan broj instrukcija po funkcijskoj točki
1. generacija jezika	1	320
2. generacija jezika	3	107
3. generacija jezika	4	80
4. generacija jezika	16	20
5. generacija jezika	70	5

Tablica 2.1 : Programski jezici i razine

Prema većini istraživanja, razina programskog jezika doprinosi produktivnosti programiranja, tako Jones navodi slijedeće rezultate (Jones, 1996., str 1.):

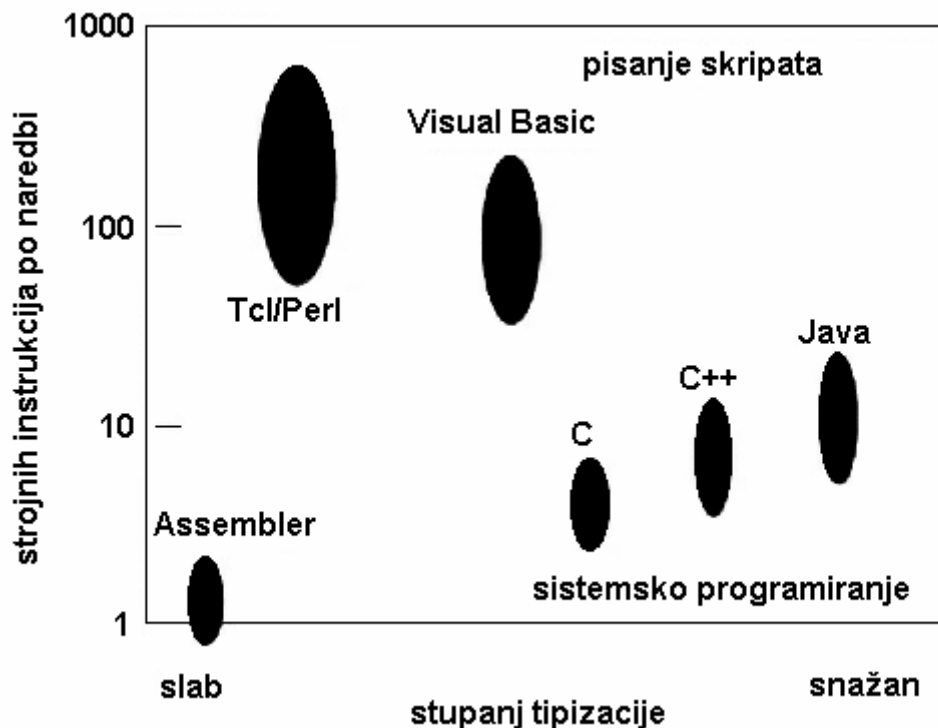
Razina jezika (broj strojnih naredbi po instrukciji)	Prosječna produktivnost po osobi mjesečno (u broju funkcijskih točaka)
1 - 3	5 - 10
4 - 8	10 - 20

9 - 15	16 - 23
16 - 23	15 - 30
24 - 55	30 - 50
više od 55	40 - 100

Tablica 2.2 : Odnos razine jezika prema produktivnosti programera

7.3.2.2 Nizak stupanj tipizacije

Tipizacija predstavlja stupanj u kojem je značenje informacije specificirano prije njenog korištenja. U strogo tipiziranom jeziku programer definira kako će se koristiti svaki dio informacije i prevoditelj sprečava njeno korištenje na bilo koji drugi način. U slabo tipiziranom jeziku nema unaprijed zadanih restrikcija u odnosu na način korištenja informacija – značenje informacije se definira načinom njenog korištenja.



Slika 2.1 : Usporedba različitih programskih jezika obzirom na razinu jezika i stupanj tipizacije (Ousterhout,1998.)

Slijedeći primjer riješen je u dva programska jezika, C++ kao primjeru strogo tipiziranog programskog jezika i Perl-u, kao primjeru slabo tipiziranog jezika skripata:

C++:

```
#include <iostream.h>
#include <string.h>
int a;           // cjelobrojni podatak
char b[30];      // niz znakova
void main () {
    a = 10;
```

```
cout << "a = " << a << endl; // ispis vrijednosti varijable a
strcpy (b,"niz znakova");
cout << "Varijabla b sadrži " << b << "." << endl; }
```

Perl:

```
$a = 10;
print "a = $a\n";
$a = "niz znakova";
print "Varijabla a sadrži $a.\n";
```

Usporedbom ova dva rješenja možemo vidjeti da u slabo tipiziranom programskom jeziku kao što je Perl istu varijablu možemo koristiti na različite načine, tako da npr. jednom sadrži numeričku vrijednost, a drugi put niz znakova, što kod strogo tipiziranog programskog jezika, kao što je C++ nije moguće. Također, varijabla u Perl-u uopće nije deklarirana prije upotrebe.

7.3.3 Generiranje izvještaja u obliku html-a pomoću jezika Perl, na primjeru studentske oglasne ploče

7.3.3.1 Podaci

RDBMS: Postgresql
Baza: darados

Struktura tablice oglasa:

polje	tip	opis
r_broj	integer	redni broj sloga
vrijeme	varchar	sistemska datum i vrijeme
rubrika	varchar	rubrika i podrubrika oglasa
lozinka	varchar	lozinka s kojom je oglas upisan
naslov	varchar	naslov oglasa
tekst	varchar	tekst oglasa
telefon	varchar	telefon oglašivača
mail	varchar	e-mail oglašivača

7.3.3.2 Predlošci za generiranje oglasa

oglas.predlozak1 (uključuje telefon oglašivača):

```
<p>#r_broj. #vrijeme #rubrika</p>
<p><b>#naslov</b></p>
<p>#tekst</p>
<p>Telefon:#telefon<br>E-mail:#mail</p>
<hr>
```

oglas.predlozak2 (bez telefona):

```

<p>#r_broj. #vrijeme #rubrika</p>
<p><b>#naslov</b></p>
<p>#tekst</p>
<p>E-mail:#mail</p>
<hr>

```

7.3.3.3 Ulazni parametri korisnika (rubrika i podrubrika; html):

- prema njima se kreira upit u SQL bazu

```

<html>
<head>
<title>Oglasi po grupama</title>
<meta http-equiv="Content-Type" content="text/html; charset=windows-1250">
</head>
<body bgcolor="#339977">

  <table width="100%" border="0" cellpadding="5" bgcolor="#FFFFFF">
    <tr>
      <td>
        <div align="center"><font size="+2" color="#FF0000" face="Verdana,
        Arial, Helvetica, sans-serif"><b><a href="index.html">Oglasi
        po grupama</a></b><br>
        </font></div>
      </td>
    </tr>
  </table>
  <p>&nbsp;</p>
  <div align="center">
    <form name="form1" action="http://student.foi.hr/~darados/oglas_iispis.cgi" method="POST" >
      <table border="0" cellpadding="5">
        <tr>
          <td>&nbsp;</td>
          <td><b>&nbsp;</b></td>
        </tr>
        <tr>
          <td>Rubrika:
            <select name="rubrika">
              <option value="ponuda">Ponuda</option>
              <option value="potražnja">Potražnja</option>
            </select>
            Odnosi se na:
            <select name="predmet">
              <option value="intelekt">Intelektualne usluge</option>
              <option value="skripte">Skripte i knjige</option>
              <option value="računala">Računala i programe</option>
              <option value="stanovanje">Stanovanje</option>
              <option value="ostalo">Ostalo</option>
            </select>
          </td></tr>
          <tr>
            <td><p><br>
              <input type="submit" name="Submit" value="Pregled oglasa">
            </p>
          </td></tr>
          <tr>
            <td valign="top"><font color="#FF0000"><i><br>
            <br>
            &nbsp;</td>
          </tr>
        </table>
      </form>
    <p>&nbsp;</p>
  </div>
</body>

```

forma za unos podataka;
pritiskom na tipku "submit"
pokreće se oglas_iispis.cgi

parametar "rubrika"

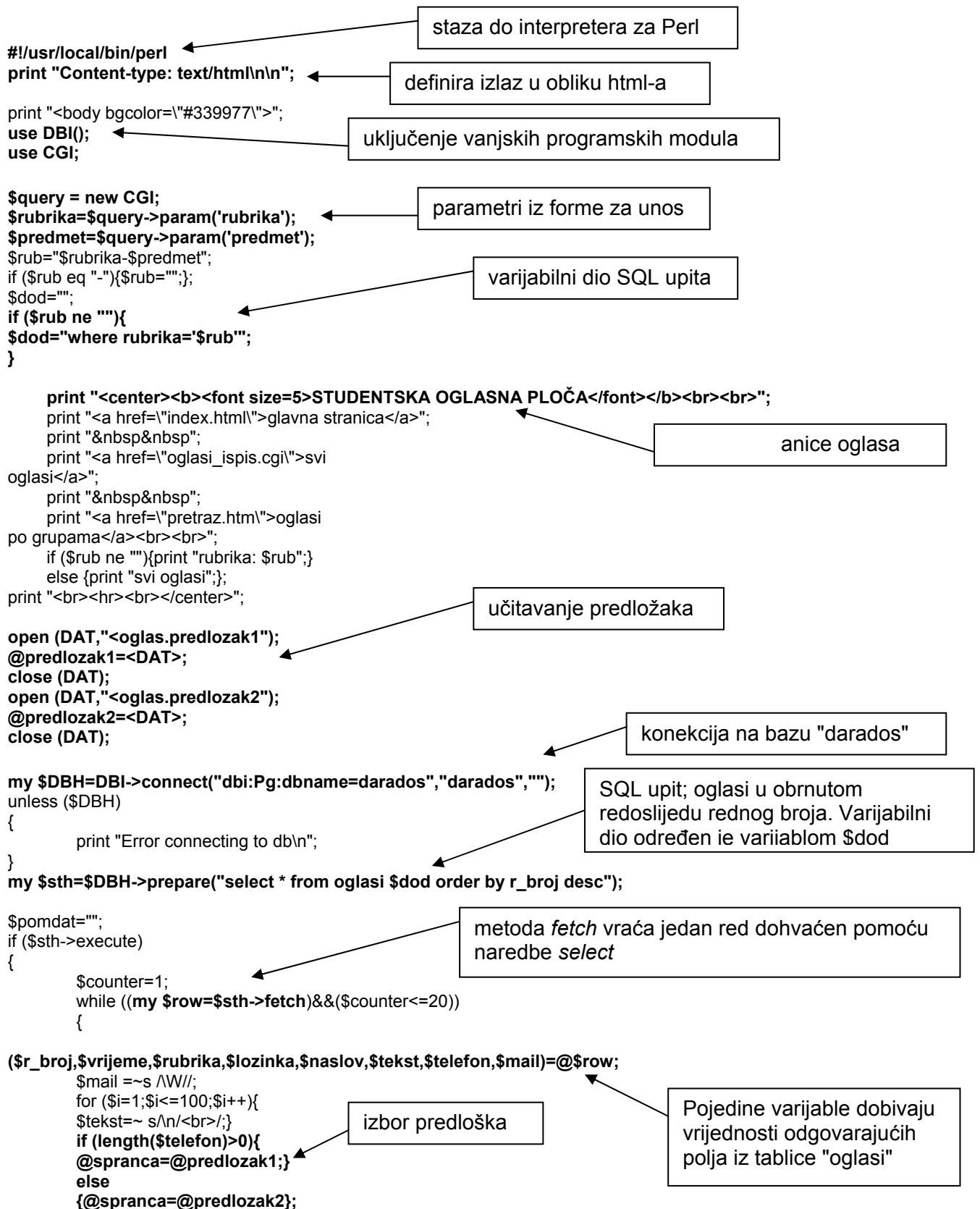
parametar "predmet"

tipka "submit"

kraj forme za unos podataka

</html>

7.3.3.4 Generiranje stranice oglasa (kod u Perl-u)



```

$veza="<a href='mailto:$mail\@foi.hr'>$mail\@foi.hr</a>";
foreach $line (@spranca){
  $line=~ s/#r_broj/$r_broj/;
  $line=~ s/#rubrika/$rubrika/;
  $line=~ s/#telefon/$telefon/;
  $line=~ s/#vrijeme/$vrijeme/;
  $line=~ s/#mail/$veza/;
  $line=~ s/#naslov/$naslov/;
  $line=~ s/#tekst/$tekst/;
  print "$line";}
  $counter++;
}
}
$sth->finish;

$DBH->disconnect;

#open (DAT,"<obavijesti.pod");
#@redovi=<DAT>;
#close (DAT);
print "<center><font color=##ff0000>Svu odgovornost za sadržaj oglasa snose
isključivo oglašivači!</font></center><br>";
foreach $line (@redovi){
  print $line;}

```

ake iz predloška
odgovarajućom vrijednoću varijable

ispis formatiranih podataka na web stranicu

diskonektiranje s baze

pis podnožja stranice

7.3.3.5 Izlazni html dokument

```

<body bgcolor="#339977"><center><b><font size=5>STUDENTSKA OGLASNA
PLOČA</font></b><br><br><a href="index.html">glavna stranica</a>&nbsp;<a href="oglas_i_ispis.cgi">svi
oglas</a>&nbsp;<a href="pretraz.htm">oglas</a>&nbsp;<a href="potraznja-ostalo">potražnja-ostalo<br><br></center>
<p>211. Fri Jan 12 07:42:19 CET 2001
potražnja-ostalo</p>
<p><b>Proba.....</b></p>
<p>Ovo je samo probni oglas!</p>
<p>E-mail:<a href="mailto:darados@foi.hr">darados@foi.hr</a></p>
<hr>
<p>208. Thu Jan 11 13:38:18 CET 2001
potražnja-ostalo</p>
<p><b>Traži se skripta iz statistike</b></p>
<p>Traži se skripta iz statistike. Hitno!</p>
<p>E-mail:<a href="mailto:darados@foi.hr">darados@foi.hr</a></p>
<hr>
<p>199. Thu Dec 14 10:17:26 CET 2000
potražnja-ostalo</p>
<p><b>Oglas...</b></p>
<p>Ovo je samo proba!</p>
<p>E-mail:<a href="mailto:darados@foi.hr">darados@foi.hr</a></p>
<hr>
<center><font color=##ff0000>Svu odgovornost za sadržaj oglasa snose
isključivo oglašivači!</font></center><br>

```

oglas

podnožje stranice

7.4 C#

C# (čita se: See Sharp) je novi Microsoftov programski jezik, dio programskog paketa Visual Studio 7, namijenjen ponajprije izradi mrežnih aplikacija za Internet. Sintaksa je u najvećem dijelu preuzeta od C++, to je objektno-orijentirani programski jezik koji nastoji ujediniti efikasnost C++ sa visokom razinom i produktivnošću Visual Basica.

Osnovne novine koje C# uvodi u odnosu na C++ su slijedeće :

- veća sigurnost rukovanja tipovima (po uzoru na Pascal) : nemoguće je varijabli dodijeliti vrijednost varijable nekog drugog tipa bez konverzije tipa (upotrebe *cast* operatora)

Primjer:

```
char c;  
int i=65;  
c=i;    // radi u C++, a nije dopušteno u C#  
c= (char) i;
```

- rukovanje logičkim tipom podataka (bool) je također po uzoru na Pascal, odnosno, nema standardne konverzije između tipa *bool* i ostalih tipova (Hejlsberg, Wiltamuth, 2000, str. 63.). To znači da su svi logički izrazi egzaktno logičkog tipa, umjesto cjelobrojnog, kao što je to slučaj kod C i C++.

- automatska dealokacija nerefenciranih dijelova memorije (automatic garbage collection). Ova osobina bitno pridonosi produktivnosti programiranja, jer programer više ne mora svaki alocirani memorijski prostor oslobađati naredbama *free*, odnosno *delete*.

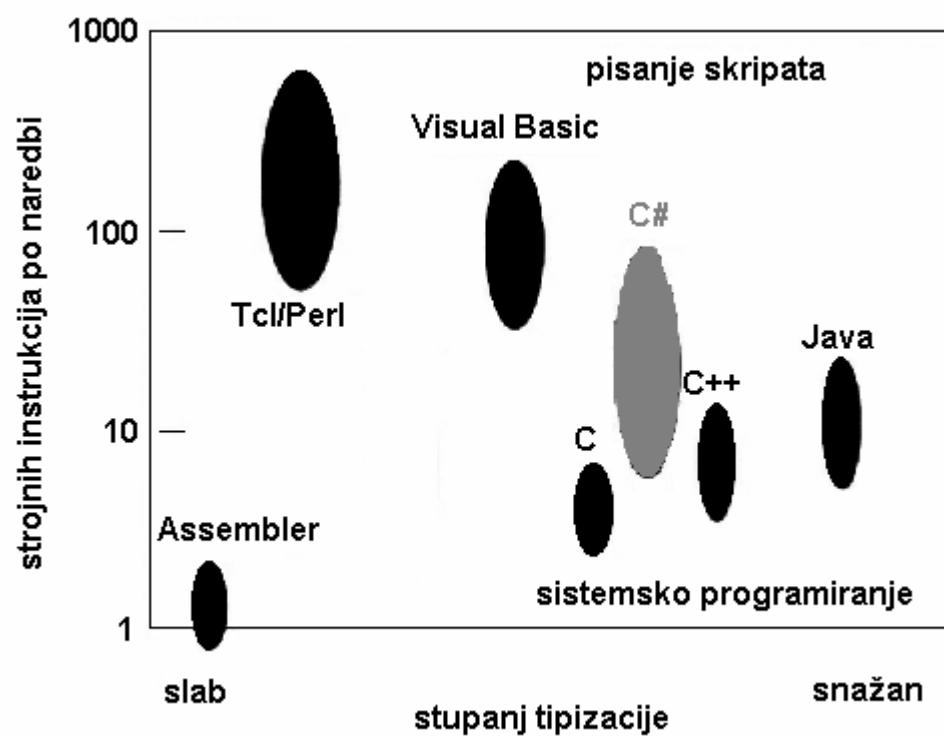
- iteracija *foreach*. Ta iteracija prisutna je u nekim jezicima skripata, kao što je Perl, a omogućuje obradu skupnih tipova podataka, kao što su vektori. Svi vektori izvedeni su iz apstraktne klase *System.Array*, koja definira i metode za navigaciju unutar polja, popunjavanje polja i sl.

- pojednostavljena manipulacija znakovnim nizovima (po uzoru na Pascal). Znakovnim nizovima moguće je pridruživati vrijednosti pomoću operatora pridruživanja vrijednosti (=), odnosno uspoređivati ih pomoću relacijskih operatora, umjesto korištenja funkcija iz biblioteke *string.h*.

- delegati. Delegati u izvjesnom smislu predstavljaju zamjenu za pokazivače, omogućujući međutim, veću sigurnost programskog koda, budući da pokazivači omogućuju i dohvaćanje dijelova memorije koje ne alocira program. Dobro svojstvo delegata je da može referencirati objekt iz bilo koje klase.

Na temelju gore navedenog, može se zaključiti da jezik C# predstavlja iskorak u smislu veće sigurnosti programskog koda, bolje kontrole i pojednostavljenja programiranja. U tom smislu uvedena su neka svojstva Pascala. Sve to rezultira visokom razinom jezika i većom fleksibilnošću u odnosu na C++, što C# približava jezicima skripata.

U izvjesnom smislu, jezik C# predstavlja i "kariku koja nedostaje" u razvoju programskih jezika, približavajući objektno-orijentirano programiranje jezicima skripata, tako da bi smo ga među programskim jezicima sa slike 2.1 (Ousterhot, 1998., str. 25) mogli pozicionirati kao na slijedećoj slici :



Slika 2.2 : C# popunjava prazninu između sistemskih objektno-orijentiranih programskih jezika i jezika skripata